

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3– Mock Interview

Course Code:	ITA0512	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	S. Susannal Devapriyam	Reg. No.:	192525398
Section / Batch:	B. Tech AI & ML	Date:	26 - 08 - 2026

Code of Conduct

I, S. Susannal Devapriyam (192525398) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Susannal Devapriyam

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

A surveillance system must highlight moving objects in real time. Design a lightweight solution, explaining your approach and optimization strategies.

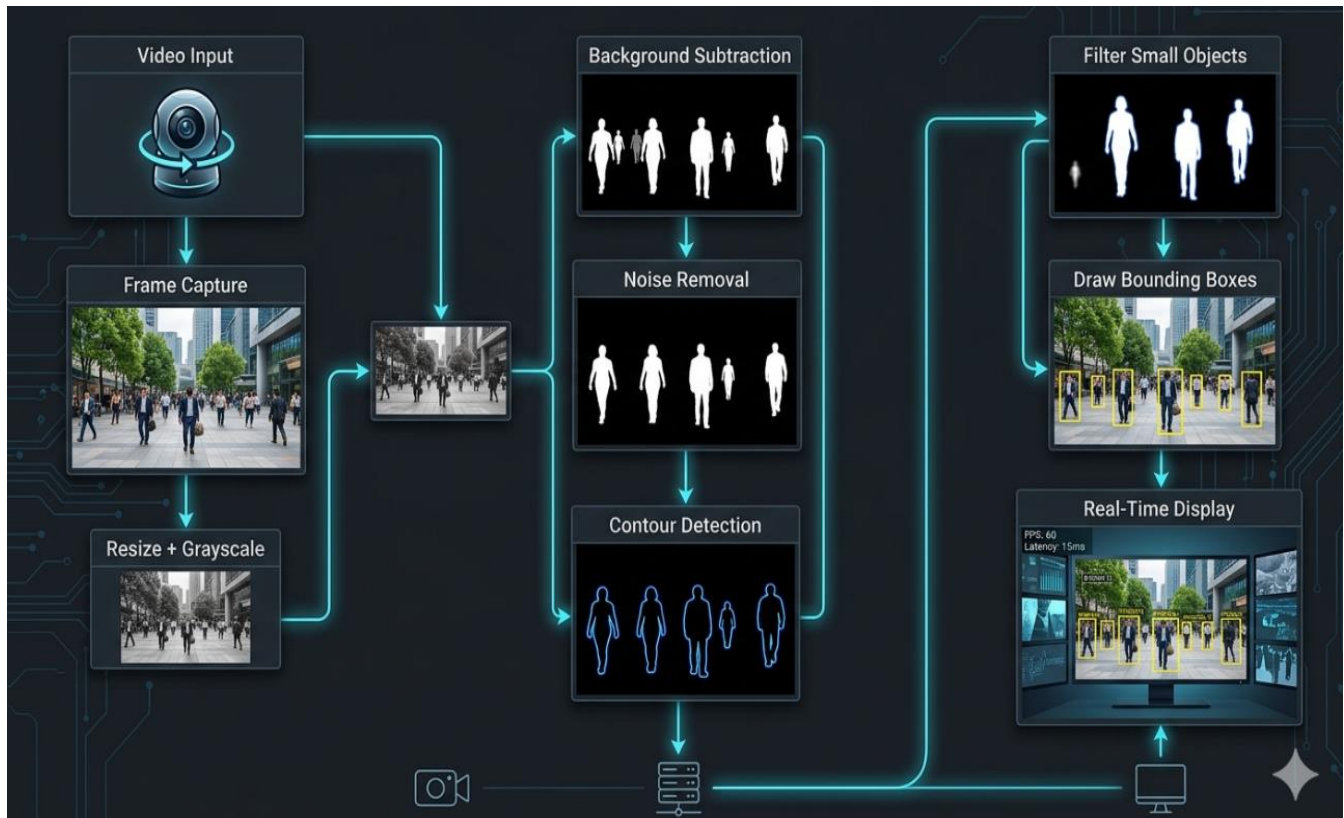
1. Proposed Solution

For a lightweight real-time surveillance system, I would use Background Subtraction combined with contour detection.

Instead of using computationally expensive deep learning models, the system compares each video frame with a continuously updated background model. The changed regions are considered foreground objects. Noise is removed using morphological operations, and contours are used to identify significant moving objects.

Pipeline

2. Why This Approach?



I selected background subtraction because surveillance cameras are usually fixed, making it easier to distinguish moving objects from a relatively stable background.

Advantages:

- Lightweight and suitable for real-time processing.
- Requires less computational power than deep learning models.
- Works efficiently on ordinary CPUs.
- Detects multiple moving objects simultaneously.
- Easy to implement using OpenCV.

For dynamic backgrounds, OpenCV's **MOG2 Background Subtractor** can adaptively update the background model.

3. Python Implementation

```
import cv2

# Open webcam or surveillance video
cap = cv2.VideoCapture(0)

# Create adaptive background subtractor
background_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=50,
    detectShadows=False
)

while True:
    ret, frame = cap.read()

    if not ret:
        break

    # Resize frame for faster processing
    frame = cv2.resize(frame, (640, 480))

    # Apply background subtraction
    foreground_mask = background_subtractor.apply(frame)

    # Remove noise using morphological operations
    kernel = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (5, 5)
    )
```

```

foreground_mask = cv2.morphologyEx(
    foreground_mask,
    cv2.MORPH_OPEN,
    kernel
)

# Find moving object contours
contours, _ = cv2.findContours(
    foreground_mask,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:

    # Ignore small noisy regions
    if cv2.contourArea(contour) < 500:
        continue

    # Get bounding box
    x, y, w, h = cv2.boundingRect(contour)

    # Highlight moving object
    cv2.rectangle(
        frame,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )

    cv2.putText(
        frame,
        "Moving Object",
        (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 0),
        2
    )

# Display output
cv2.imshow("Real-Time Surveillance", frame)

```

```
cv2.imshow("Foreground Mask", foreground_mask)

# Press Q to exit
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

4. Working of the Solution

Step 1: Video Capture

The system continuously captures frames from a webcam or surveillance camera.

Step 2: Frame Resizing

Each frame is resized to 640×480 pixels to reduce processing time.

Step 3: Background Subtraction

MOG2 compares the current frame with the learned background and extracts moving regions.

Step 4: Noise Removal

Morphological opening removes small unwanted regions from the foreground mask.

Step 5: Contour Detection

Contours identify connected regions representing possible moving objects.

Step 6: Object Filtering

Very small contours are ignored using a minimum area threshold.

Step 7: Object Highlighting

Bounding boxes and labels are drawn around significant moving objects.

5. Optimization Strategies

For real-time performance, I would apply the following optimizations:

1. Resize Input Frames

Processing smaller frames significantly reduces the number of pixels and computational cost.

2. Avoid Deep Learning for Simple Motion Detection

Background subtraction is faster and more suitable when object classification is unnecessary.

3. Filter Small Contours

Ignoring small regions prevents unnecessary processing caused by sensor noise.

4. Use Efficient OpenCV Functions

OpenCV provides optimized implementations for background subtraction and image processing.

5. Process Only Important Regions

After detecting motion, further processing can be limited to regions containing foreground objects.

6. Reduce Frame Processing Frequency

If required, the system can process alternate frames to improve speed on low-end devices.

6. Handling Practical Challenges

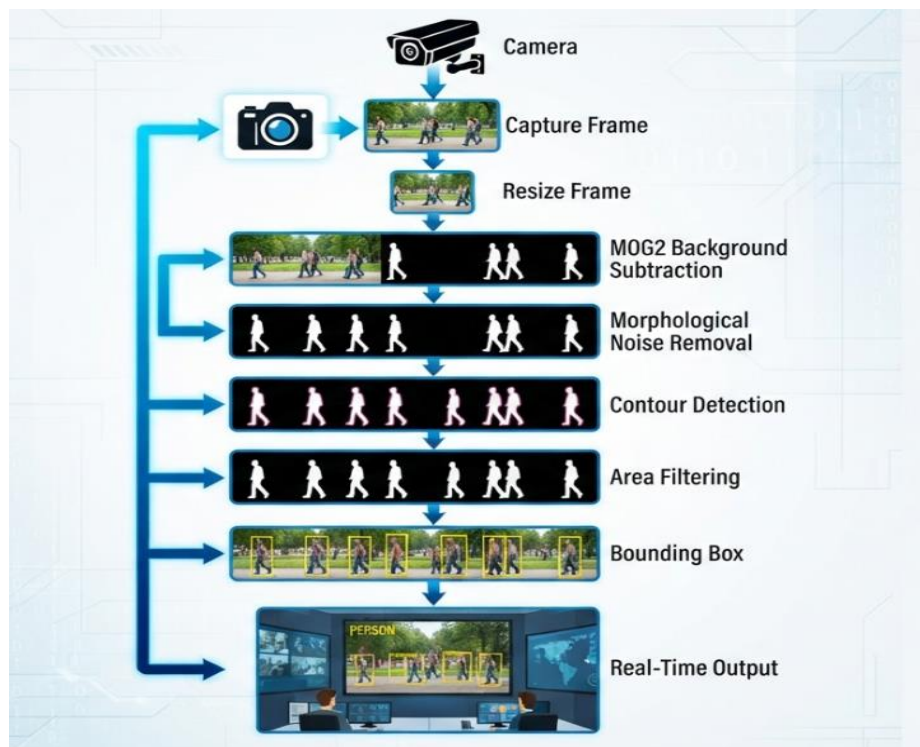
Challenge	Solution
Camera noise	Morphological filtering
Dynamic background	Adaptive MOG2 background model

Small false detections	Minimum contour area
Slow processing	Frame resizing
Shadows	Disable shadow detection
Multiple objects	Process multiple contours

8. Conclusion

The proposed solution uses adaptive background subtraction, noise filtering, and contour detection to highlight moving objects in real time. It is lightweight, computationally efficient, and suitable for surveillance systems running on ordinary hardware. Frame resizing and contour filtering further improve real-time performance while maintaining reliable moving-object detection.

Simple Architecture Diagram:



Output:

